

This is an excellent compilation of frontend interview questions! Let me categorize everything and create a comprehensive document for you.

Frontend Interview Questions - Complete Guide

Table of Contents

1. [HTML](#html)
2. [CSS](#css)
3. [JavaScript](#javascript)
4. [React](#react)
5. [Next.js](#nextjs)
6. [Redux & State Management](#redux--state-management)
7. [Browser Internals & Performance](#browser-internals--performance)
8. [Scenario-Based Questions](#scenario-based-questions)
9. [Machine Coding Challenges](#machine-coding-challenges)
10. [Data Structures & Algorithms](#data-structures--algorithms)
11. [System Design](#system-design)
12. [Security](#security)
13. [Accessibility](#accessibility)
14. [Testing & Debugging](#testing--debugging)

HTML

Basic Questions

1. What is the difference between `id` and `class` attributes?
2. How does the `doctype` declaration affect a webpage?
3. Explain semantic HTML and its importance
4. What are `meta` tags, and why are they used?
5. What is the difference between `span` and `div` tags?
6. What is the difference between inline, block, and inline-block elements?
7. What are custom data attributes, and how are they used?

Advanced Questions

8. How does the `defer` attribute in a script tag work?
9. What is `async` vs `defer` in JS?
10. How do you optimize images for web performance in HTML?
11. How would you create an accessible custom tooltip using only HTML?
12. Why do we still need `<!DOCTYPE html>`?
13. What happens if the browser enters quirks mode?
14. How do browsers parse HTML, CSS, and JavaScript?

CSS

Basic Questions

1. What is the CSS box model?
2. Explain the difference between `margin` and `padding`
3. What are CSS selectors and their specificity?
4. What is the difference between `position: relative`, `absolute`, `fixed`, and `sticky`?
5. How does flexbox work?
6. How does CSS Grid differ from Flexbox?

Advanced Questions

7. Explain the Critical Rendering Path
8. How does CSS affect rendering performance?
9. Difference between reflow vs repaint
10. How do you optimize CSS delivery?
11. A CSS animation is janky on mobile devices—how do you identify and fix the issue?
12. How would you implement a responsive design without media queries?
13. What are CSS custom properties (variables) and their use cases?

Layout Challenges

14. Holy Grail Layout implementation
15. How would you handle UI breaking due to:
 - Long text
 - Large images
 - Extremely slow networks

JavaScript

Core Fundamentals

1. Why do closures exist in JavaScript?
2. How does the event loop actually work?
3. What's the difference between `null` and `undefined`?
4. Why does `this` behave differently in arrow functions?
5. What happens when you compare objects with `==` vs `===`?
6. Why can you call a function before it's declared?
7. What's the difference between `call`, `apply`, and `bind`?
8. Explain the difference between `var`, `let`, and `const`
9. What is "hoisting" in JavaScript?
10. How does prototypal inheritance work?

Advanced Concepts

11. What are stale closures in JS?
12. What is IIFE? Use case of IIFE?
13. Explain the concept of the execution context
14. What is the Temporal Dead Zone?
15. Difference between ES6 classes and constructor functions
16. Pass by Value vs Reference
17. Primitive vs Reference Types
18. Type Coercion and Truthy/Falsy Values

Asynchronous JavaScript

19. Callbacks vs Promises vs Async/Await
20. What are promises, and how do they compare to async/await?
21. How does Promise chaining work?
22. Explain Promise.all, Promise.race, Promise.allSettled, Promise.any
23. Error handling in async operations
24. Microtasks vs Macrotasks
25. Output-based questions on promises, setTimeout, and "this" keyword

DOM & Events

26. Event flow in the DOM: Capturing vs bubbling
27. `preventDefault` vs propagation control
28. Scenario-based questions on event bubbling & event delegation
29. How does event delegation work and why is it important?

Performance & Optimization

30. What is debouncing and throttling in JavaScript?
31. Scenario Questions on Debouncing/throttling/prefetch/preload
32. What causes memory leaks in JavaScript, and how do you prevent them?
33. Memoization techniques

Polyfills (Very Important!)

34. Implement Promise.all
35. Implement Promise.race
36. Implement Promise.any
37. Writing debounce() & throttle() from scratch
38. Implementing your own map, filter, reduce
39. Write a polyfill for .filter()
40. Polyfilling bind, call, apply
41. Custom Deep Copy utility (without lodash)

Advanced Patterns

42. Designing a Pub/Sub Event Emitter
43. Factory Functions vs Constructor Functions
44. Custom Promise implementation
45. Observables (Basic intro for high-end companies)

React

Fundamentals

1. What is React and how does it work?
2. What are the differences between functional and class components?
3. What are props and state? How are they different?
4. What is JSX, and why is it used in React?
5. What is the Virtual DOM, and why is it important?

6. What is the purpose of the key prop in React lists?
7. How do you handle events in React?
8. What are default props in React?
9. What is conditional rendering in React?

Hooks

10. What are React Hooks? Explain useState and useEffect with examples
11. What is the difference between useMemo and useCallback?
12. What happens when you update state inside useEffect?
13. useEffect vs useState
14. useTransition vs useDeferredValue
15. How do custom hooks improve scalability and reusability?
16. Building a simple useState hook (for interviews)
17. Custom hooks anti-patterns
18. Ref forwarding and imperative handles

Component Patterns

19. What is React.memo, and how does it help with performance optimization?
20. What is a Higher-Order Component (HOC), and how is it used?
21. Compound components pattern
22. Render props vs HOCs - when to use what
23. What are render props, and how are they different from HOCs?
24. Controlled vs uncontrolled components

State Management

25. What is the Context API, and when should you use it instead of Redux?
26. What is prop drilling, and how can it be avoided?
27. How do you manage state in a complex app to avoid unnecessary re-renders?
28. How do you share state between components without prop drilling or context?

Performance Optimization

29. How does React handle re-renders, and how can you optimize unnecessary renders?
30. What is reconciliation in React?
31. How does React's diffing algorithm work?
32. Why does React.memo not work when you pass objects as props?
33. How do you optimize a React application rendering 100k+ items in a list?
34. Move state down, lift content up, split components
35. Memoization: useMemo, useCallback, React.memo - when to use each
36. Why does your component re-render even when state value is the same?
37. Virtualization Concepts (Windowing Lists)

Advanced Concepts

38. What is React.lazy and Suspense? How does lazy loading work in React?
39. What are error boundaries, and how do they work?
40. React Server Components explained
41. Suspense boundaries and streaming SSR
42. Concurrent rendering mental model
43. How does React Fiber work internally?

44. What are React Fiber and Concurrent Mode?

45. React Suspense for async loading

Forms & Validation

46. How does React handle forms, and what are controlled inputs?

47. Difference between controlled and uncontrolled components in forms

48. How would you manage form validation, error handling, and state for dynamic inputs?

Routing & Navigation

49. What is React Router, and how does client-side routing work?

50. How do you protect routes that require authentication?

Testing

51. How do you test React components? What are commonly used testing libraries?

52. How do you test React hooks and async logic?

Common Pitfalls

53. Why should you avoid using array index as key in dynamic lists?

54. What happens if you forget the dependency array in useEffect?

55. What happens when you call setState inside useEffect without dependencies?

56. Why should you avoid setting state based on previous state directly?

57. What happens if you update state during render?

Next.js

Rendering Strategies

1. Explain the difference between SSR, SSG, ISR, and CSR in Next.js

2. Where would you choose each rendering strategy in a real-world scenario?

3. What happens if you overuse getServerSideProps in a high-traffic app?

4. How does ISR (Incremental Static Regeneration) really work under the hood?

5. When would SSR help, and when would it hurt?

6. How does server-side rendering (SSR) differ from client-side rendering (CSR)?

Architecture & Routing

7. How would you structure a large-scale Next.js app with nested routes, dynamic routes, and API routes?

8. How do Next.js Middlewares differ from API routes?

9. When would you run logic at the Edge instead of using SSR?

Performance & Optimization

10. How do you manage caching in a Next.js + CDN setup, especially with SSR pages?

11. What are the trade-offs of using Next.js next/image vs a third-party CDN?

12. What tools do you use in Next.js to detect and reduce large bundle sizes?

13. You shipped a Next.js app and users complain about slow first paint. Walk through your debugging strategy.

SEO & Metadata

14. How would you handle SEO for dynamically generated pages in Next.js?
15. How do you ensure crawlers see the right metadata without hurting performance?

Security & API Routes

16. How do you secure API routes in Next.js when handling sensitive user data?

Debugging

17. Ever debugged hydration mismatch errors? How do you approach them?

Migration

18. How would you justify migrating an old React SPA into Next.js to a skeptical CTO?

Redux & State Management

Redux Basics

1. Redux vs React Context API — when to use each?
2. Why did you choose Redux over Context API?
3. Atomic state (Jotai/Recoil) vs Redux
4. When NOT to use global state

Advanced State Management

5. Server state vs client state separation
6. Optimistic updates patterns
7. React Query/SWR internals
8. State machines (XState) use cases
9. How do you manage derived state efficiently?
10. How do you handle cross-component communication without prop drilling?
11. What's your strategy for managing global state in a large React app with multiple teams?
12. How do you manage real-time state across multiple browser tabs?

Browser Internals & Performance

Browser Fundamentals

1. Critical rendering path and reflow/repaint
2. How do browsers parse HTML, CSS, and JavaScript?
3. What actually blocks rendering, and why?
4. DNS resolution and browser caching
5. HTML parsing and DOM construction
6. CSS blocking vs JavaScript blocking
7. How the browser actually paints pixels

Caching & Storage

8. Browser caching strategies (304 vs fresh)

9. How does browser caching work?
10. LocalStorage vs SessionStorage vs Cookies vs IndexedDB
11. Cookies vs browser storage, when and why?
12. Storage handling using LocalStorage / SessionStorage / IndexedDB

Service Workers & Web Workers

13. Service workers and offline strategies
14. Web Workers for heavy computation
15. Web Workers (Basics)
16. How do service workers improve web performance?

Network & APIs

17. How CORS actually works
18. Why is CORS needed, and how does it work?
19. HTTP/2 vs HTTP/3 implications for FE
20. WebSockets vs Polling (Basic Differences)
21. Questions on API calls (Retry Mechanism + Cancelling old API calls)

Performance Metrics

22. Core Web Vitals (LCP, INP, CLS)
23. Which frontend performance metrics do you monitor in production?
24. What are critical rendering paths?
25. How do you optimize for Time to First Byte (TTFB)?
26. RUM vs synthetic monitoring

Optimization Techniques

27. How would you investigate and improve slow initial page load?
28. Ways to optimize assets before touching application code
29. How do CDNs affect perceived performance?
30. CDN Integration for static assets
31. What concrete steps would you take to reduce initial load time by ~40%?

Build Tools & Bundling

32. Tree shaking - what actually gets removed
33. Vite vs Webpack vs Turbopack
34. Module federation architecture
35. Source maps in production (pros/cons)
36. Bundle analysis and optimization
37. Dynamic imports done right
38. Bundle Size Optimization
39. How do you reduce the size of a JavaScript bundle?

Scenario-Based Questions

Performance Issues

1. A page loads but renders nothing, where do you start debugging?

2. A data-heavy dashboard slows down when filters are applied — how do you profile and optimize it?
3. You notice a memory leak in a production SPA—how do you identify and fix it?
4. If you suspect a memory leak in a React application, how would you identify and resolve it?
5. How would you debug a performance bottleneck in a React app using DevTools?
6. Production debugging without source maps

API & Data Handling

7. How do you approach slow or unreliable APIs?
8. How would you design a frontend system that handles caching, retries, and error boundaries gracefully?
9. How do you handle real-time updates in a React application efficiently?

UI/UX Issues

10. Users report intermittent UI glitches in different browsers—how would you troubleshoot?
11. A critical UI feature is failing during peak traffic—how do you mitigate the issue?
12. How do you ensure consistent UI rendering across browsers, screen sizes, and devices?

Code Quality & Architecture

13. How do you structure a large React codebase so it stays scalable, readable, and easy to maintain?
14. You need to migrate a legacy frontend codebase to a modern framework—what's your plan?
15. A component breaks when upgrading a library version—how do you manage dependencies?
16. How do you ensure maintainable and scalable code?

Real-World Debugging

17. Describe a production issue caused by your frontend code. How did you respond and fix it?
18. What broke in production and how you fixed it?

Team & Process

19. What makes a good Pull Request review?
20. How do you balance readability vs optimization?
21. How do you explain frontend performance metrics like FCP, TTI, or CLS to non-technical stakeholders?

Machine Coding Challenges

Basic Components

1. Todo List
2. Star Rating
3. Tabs Component
4. Accordion

5. Modal/Dialog
6. Dropdown
7. Light/Dark Theme Toggle

Forms & Input

8. Config Driven Form (System Design)
9. Dynamic form with field-level validation
10. Multi-step form with state management
11. Search functionality with live filtering
12. Autocomplete/Typeahead (System Design)
13. Input search box with API calls (debouncing + async handling)

Lists & Data Display

14. Pagination with JS
15. Truncated Pagination with React
16. Infinite Scroll
17. Infinite scrolling implementation
18. Virtualized list for large datasets
19. E-commerce Filters
20. Posts with Comments
21. Nested Comment Section

Interactive UI

22. Carousel (System Design)
23. Draggable list implementation
24. Advanced Tic Tac Toe
25. Match Similar Tiles
26. Configurable Color Boxes

Layout

27. Holy Grail Layout
28. Responsive grid system

Notifications & Feedback

29. Toast/Notification Component (System Design)
30. Progress Bar
31. Global toast with queue management

Complex Features

32. Shopping Cart
33. Poll Widget (System Design)
34. File upload with progress, chunking, and resume
35. Image-heavy feed with lazy loading

Custom Hooks

36. Custom useFetch hook for HTTP requests
37. Custom hook to debounce user input
38. Custom hooks for various use cases

Data Structures & Algorithms

Arrays and Strings

Basic Array Problems

1. Find the maximum sum subarray (Kadane's Algorithm)
2. Find the missing number in an array of integers
3. Merge two sorted arrays into one sorted array
4. Remove duplicates from a sorted array
5. Rotate Array
6. Sort Array by Parity
7. Sort Array by Parity II
8. Rearrange Array Elements by Sign
9. Remove Element
10. Apply Operations to an Array
11. Find All K-Distant Indices in an Array
12. Product of array elements except the current index
13. Flatten a nested array

Two Pointer Problems on Arrays

14. Two Sum → <https://leetcode.com/problems/two-sum/>
15. 3Sum → <https://leetcode.com/problems/3sum/>
16. 3Sum Closest → <https://leetcode.com/problems/3sum-closest/>
17. 4Sum → <https://leetcode.com/problems/4sum/>
18. Container With Most Water → <https://leetcode.com/problems/container-with-most-water/>
19. Sort Colors → <https://leetcode.com/problems/sort-colors/>
20. Watering Plants II → <https://leetcode.com/problems/watering-plants-ii/>
21. Next Permutation → <https://leetcode.com/problems/next-permutation/>
22. Next Greater Element III → <https://leetcode.com/problems/next-greater-element-iii/>
23. Partition Array According to Given Pivot
24. Merge Two 2D Arrays by Summing Values
25. Merge Sorted Array

String Problems

26. Find all substrings that are palindromes
27. Check if a string is a palindrome
28. Find the first non-repeating character in a string
29. Reverse String → <https://leetcode.com/problems/reverse-string/>
30. Reverse Prefix of Word
31. Reverse Vowels of a String
32. Reverse Words in a String
33. Reverse Words in a String III
34. Valid Palindrome → <https://leetcode.com/problems/valid-palindrome/>
35. Valid Palindrome II
36. Lexicographically Smallest Palindrome

37. String transformation problems with multiple constraints

Two Pointer on Strings

- 38. Merge Strings Alternately
- 39. Largest Merge of Two Strings
- 40. Shortest Distance to a Character
- 41. DI String Match
- 42. Make String a Subsequence Using Cyclic Increments
- 43. Count Binary Substrings
- 44. Minimum Length of String After Deleting Similar Ends
- 45. String Compression
- 46. Separate Black and White Balls
- 47. Move Pieces to Obtain a String
- 48. Sentence Similarity III

Linked Lists

Basic Operations

- 49. Reverse a linked list
- 50. Detect a cycle in a linked list
- 51. Find the middle of a linked list
- 52. Merge two sorted linked lists
- 53. Implement a stack using a linked list
- 54. Find the intersection point of two linked lists

Stacks and Queues

Stack Problems

- 55. Implement a stack using an array
- 56. Implement a stack that supports push, pop, top, and retrieving the minimum element
- 57. Design a max stack that supports push, pop, top, and retrieve maximum elements
- 58. Valid Parentheses
- 59. Next Greater Element
- 60. Daily Temperatures

Queue Problems

- 61. Implement a circular queue
- 62. Design a queue using stacks
- 63. Sliding Window Maximum

Trees and Binary Search Trees

Tree Traversal

- 64. Find the height of a binary tree
- 65. Implement an inorder traversal of a binary tree
- 66. Implement preorder traversal
- 67. Implement postorder traversal
- 68. Level order traversal

Tree Problems

- 69. Find the lowest common ancestor of two nodes in a binary tree
- 70. Validate if a binary tree is a valid binary search tree
- 71. Serialize and deserialize a binary tree
- 72. Find the diameter of a binary tree
- 73. Convert a binary tree to its mirror tree
- 74. Maximum depth of binary tree
- 75. Minimum depth of binary tree
- 76. Path Sum
- 77. Binary Tree Maximum Path Sum

Graphs

Graph Traversal

- 78. Implement depth-first search (DFS)
- 79. Implement breadth-first search (BFS)
- 80. Find the shortest path between two nodes in an unweighted graph

Graph Problems

- 81. Detect a cycle in an undirected graph using DFS
- 82. Detect a cycle in a directed graph
- 83. Check if a graph is bipartite
- 84. Find the number of connected components in an undirected graph
- 85. Find bridges in a graph
- 86. Topological Sort
- 87. Course Schedule (Prerequisites)
- 88. Number of Islands
- 89. Clone Graph

Sorting and Searching

Sorting Algorithms

- 90. Implement bubble sort
- 91. Implement insertion sort
- 92. Implement selection sort
- 93. Implement merge sort
- 94. Implement quicksort
- 95. Implement heap sort
- 96. Count the number of inversions in an array

Searching Algorithms

- 97. Implement binary search
- 98. Implement interpolation search
- 99. Find the kth smallest element in an array
- 100. Search in Rotated Sorted Array
- 101. Find First and Last Position of Element in Sorted Array
- 102. Search a 2D Matrix

Hash Tables and Hash Maps

- 103. Two Sum using HashMap
- 104. Group Anagrams
- 105. Longest Substring Without Repeating Characters
- 106. Subarray Sum Equals K
- 107. Top K Frequent Elements
- 108. Find All Anagrams in a String

Dynamic Programming

Basic DP

- 109. Fibonacci sequence using DP
- 110. Climbing Stairs
- 111. House Robber
- 112. Coin Change
- 113. Longest Common Subsequence
- 114. Longest Increasing Subsequence
- 115. Edit Distance
- 116. Maximum Subarray (Kadane's Algorithm)

Advanced DP

- 117. 0/1 Knapsack Problem
- 118. Partition Equal Subset Sum
- 119. Longest Palindromic Substring
- 120. Word Break
- 121. Decode Ways

Heaps and Priority Queues

- 122. Kth Largest Element in an Array
- 123. Top K Frequent Elements
- 124. Merge K Sorted Lists
- 125. Find Median from Data Stream
- 126. Task Scheduler

Bit Manipulation

- 127. Single Number
- 128. Number of 1 Bits
- 129. Counting Bits
- 130. Reverse Bits
- 131. Power of Two
- 132. Missing Number using XOR

Sliding Window

- 133. Maximum Sum Subarray of Size K
- 134. Longest Substring with K Distinct Characters
- 135. Minimum Window Substring
- 136. Permutation in String
- 137. Find All Anagrams in a String
- 138. Longest Repeating Character Replacement

Backtracking

- 139. Generate all permutations of an array
- 140. Generate all subsets (Power Set)
- 141. Combination Sum
- 142. N-Queens Problem
- 143. Sudoku Solver
- 144. Word Search
- 145. Palindrome Partitioning

Greedy Algorithms

- 146. Jump Game
- 147. Jump Game II
- 148. Gas Station
- 149. Partition Labels
- 150. Meeting Rooms II
- 151. Non-overlapping Intervals

Trie (Prefix Tree)

- 152. Implement Trie (Prefix Tree)
- 153. Word Search II
- 154. Design Add and Search Words Data Structure
- 155. Longest Word in Dictionary

Advanced Data Structures

Union-Find (Disjoint Set)

- 156. Number of Connected Components
- 157. Redundant Connection
- 158. Accounts Merge

Segment Tree

- 159. Range Sum Query - Mutable
- 160. Range Minimum Query

Matrix Problems

- 161. Spiral Matrix
- 162. Rotate Image

- 163. Set Matrix Zeroes
- 164. Word Search in Matrix
- 165. Longest Increasing Path in Matrix

Mathematical Problems

- 166. Multiply two numbers without using *
- 167. Power(x, n)
- 168. Sqrt(x)
- 169. Integer to Roman
- 170. Roman to Integer
- 171. Happy Number
- 172. Factorial Trailing Zeroes

Design Problems

- 173. Design a data structure that supports constant-time queries
- 174. LRU Cache
- 175. LFU Cache
- 176. Design HashMap
- 177. Design HashSet
- 178. Min Stack
- 179. Implement Queue using Stacks
- 180. Implement Stack using Queues

Miscellaneous Important Problems

- 181. Implement atoi (String to Integer)
- 182. Valid Sudoku
- 183. Trapping Rain Water
- 184. Largest Rectangle in Histogram
- 185. Maximal Rectangle
- 186. Regular Expression Matching
- 187. Wildcard Matching
- 188. Median of Two Sorted Arrays
- 189. Merge Intervals
- 190. Insert Interval

Common Patterns to Master

Pattern 1: Two Pointers

- Used for: Array/String problems, finding pairs, removing duplicates
- Time Complexity: Usually $O(n)$

Pattern 2: Sliding Window

- Used for: Substring problems, subarray problems
- Time Complexity: Usually $O(n)$

Pattern 3: Fast & Slow Pointers

- Used for: Linked list cycle detection, finding middle element
- Time Complexity: Usually $O(n)$

Pattern 4: Merge Intervals

- Used for: Overlapping intervals, scheduling problems
- Time Complexity: Usually $O(n \log n)$

Pattern 5: Cyclic Sort

- Used for: Problems with numbers in a given range
- Time Complexity: Usually $O(n)$

Pattern 6: In-place Reversal of LinkedList

- Used for: Reversing parts of a linked list
- Time Complexity: Usually $O(n)$

Pattern 7: BFS/DFS

- Used for: Tree/Graph traversal, level order problems
- Time Complexity: Usually $O(V + E)$ for graphs, $O(n)$ for trees

Pattern 8: Binary Search

- Used for: Sorted arrays, finding elements
- Time Complexity: Usually $O(\log n)$

Pattern 9: Top K Elements

- Used for: Finding top/bottom K elements
- Time Complexity: Usually $O(n \log k)$

Pattern 10: K-way Merge

- Used for: Merging sorted arrays/lists
- Time Complexity: Usually $O(n \log k)$

Pattern 11: Dynamic Programming

- Used for: Optimization problems, counting problems
- Time Complexity: Varies (usually $O(n^2)$ or $O(n)$)

Pattern 12: Backtracking

- Used for: Permutations, combinations, subsets
- Time Complexity: Usually exponential

Tips for DSA Interviews

1. ****Always clarify the problem**** - Ask about edge cases, constraints, input format
2. ****Think out loud**** - Explain your thought process
3. ****Start with brute force**** - Then optimize
4. ****Discuss time and space complexity**** - Before and after optimization
5. ****Test with examples**** - Walk through your solution with test cases
6. ****Consider edge cases**** - Empty input, single element, duplicates, negative numbers

7. ****Write clean code**** - Use meaningful variable names, proper indentation
8. ****Practice common patterns**** - Master the patterns listed above

Complexity Analysis Cheat Sheet

****Time Complexity (from best to worst):****

- $O(1)$ - Constant
- $O(\log n)$ - Logarithmic
- $O(n)$ - Linear
- $O(n \log n)$ - Linearithmic
- $O(n^2)$ - Quadratic
- $O(n^3)$ - Cubic
- $O(2^n)$ - Exponential
- $O(n!)$ - Factorial

****Space Complexity:****

- Always mention auxiliary space used
- Consider recursion stack space
- In-place algorithms use $O(1)$ extra space

System Design

Frontend System Design Questions

Real-World Applications

1. Design a News Feed (like Facebook) with infinite scroll and real-time updates
2. Build a Travel Booking website (like Airbnb) handling complex search and filtering
3. Create an E-commerce Marketplace (like Amazon) with cart persistence and checkout flow
4. Design an Email Client (like Microsoft Outlook) with offline capabilities
5. Build an Autocomplete component with advanced caching and performance optimization
6. Design a video streaming app like Netflix and YouTube
7. Design Social Media App like Twitter/Facebook
8. Design an OLX-like platform (CSR vs SSR vs SSG vs ISR)

Scalability & Architecture

9. Design a scalable dashboard with real-time data
10. Design infinite scroll for millions of users
11. Design a feature flag system
12. Design frontend architecture for a large SaaS app
13. Design a notification system (UI + state)
14. Design role-based access control on frontend
15. Design an offline-first mobile app
16. Design a design system used by 100+ teams
17. Design analytics tracking without performance loss
18. Design error handling & logging at scale

19. Design micro-frontend architecture
20. Design an app shell for fast loads
21. Design a chat UI with real-time sync
22. Design a form engine with dynamic schemas
23. Design a caching strategy for frontend
24. Design a multi-tenant frontend app
25. Design cross-platform component libraries
26. Design accessibility-first UI system
27. Design deployment & rollback strategy

Component Design

28. Design a theming system at scale
29. Design a Toast/Notification system with queue management
30. Design a Modal system
31. Design a Dropdown with search and virtualization
32. Design a Data Table with sorting, filtering, pagination
33. Design a Calendar/Date Picker
34. Design a File Upload component with progress

Performance & Optimization

35. How would you design a high-performance web app that renders large datasets with real-time updates?
36. How can your system handle 10 lakh requests per second?
37. If initial load time is high, how would you fix it?
38. How do you optimize a frontend app?

System Design Topics to Cover

- **Architecture**: Component structure, folder organization, module boundaries
- **Data Flow**: State management, API integration, caching strategy
- **Performance**: Code splitting, lazy loading, bundle optimization, CDN usage
- **Rendering**: CSR vs SSR vs SSG vs ISR
- **Real-time**: WebSockets, polling, server-sent events
- **Offline**: Service workers, IndexedDB, sync strategies
- **Security**: Authentication, authorization, XSS prevention, CSRF protection
- **Monitoring**: Error tracking, performance metrics, analytics
- **Scalability**: Micro-frontends, module federation
- **Accessibility**: WCAG compliance, keyboard navigation, screen readers
- **Testing**: Unit, integration, E2E testing strategies
- **Deployment**: CI/CD, versioning, rollback strategies

Security

Common Vulnerabilities

1. XSS prevention techniques
2. CSRF tokens and why they matter
3. How do you prevent XSS and CSRF attacks?

4. Secure token storage (JWT/OAuth)
5. Input sanitization strategies
6. Third-party script risks
7. How do you prevent common frontend security issues?
8. How do you securely handle tokens in frontend apps?

Best Practices

9. Content Security Policy (CSP)
10. HTTPS and secure cookies
11. Same-origin policy
12. CORS configuration
13. Authentication vs Authorization
14. Token-based authentication (e.g., JWT) and handling token expiry
15. Secure handling of sensitive user data on the client side

Accessibility

Fundamentals

1. ARIA roles - when to use vs avoid
2. What is ARIA, and why is it important?
3. Keyboard navigation patterns
4. Screen reader testing strategies
5. Focus management in SPAs
6. Color contrast and WCAG compliance
7. Questions on web accessibility & WCAG

Practical Implementation

8. How would you ensure a form is accessible?
9. What are some common accessibility issues in web development?
10. How do you make images accessible?
11. What is a screen reader, and how does it work?
12. How would you create an accessible navigation menu?
13. What is the difference between `role="button"` and a `` element?
14. How do you test a webpage for accessibility?
15. Accessibility considerations you actively design for
16. How do you ensure secure handling of sensitive user data on the client side?

Testing & Debugging

Testing Strategies

1. Difference between unit, integration, and E2E testing
2. How do you test React components? What are commonly used testing libraries?
3. How do you test React hooks and async logic?
4. How do you ensure maintainable and scalable code?

5. Testing: Jest, RTL, Playwright

Debugging Techniques

6. A page loads but renders nothing, where do you start debugging?
7. How would you debug a performance bottleneck in a React app using DevTools?
8. Production debugging without source maps
9. You notice a memory leak in a production SPA—how do you identify and fix it?
10. Users report intermittent UI glitches in different browsers—how would you troubleshoot?